

Applications Development Practice 4 (ADP4) - Term 2 Notes

This comprehensive study guide is structured to streamline your preparation for the upcoming Applications Development Practice 4 exam on May 27. It synthesizes the Term 2 curriculum based on the subject guidelines, covering advanced data structures (stacks, queues, trees) and a deep dive into sorting algorithms and their computational complexities.

1. Recursion and Stack Operations

Recursion and Stacks are deeply intertwined concepts. When an algorithm calls itself recursively, the operating system uses a **Call Stack** to keep track of execution contexts.

- **Last-In-First-Out (LIFO):** The foundational principle of a stack. The last element pushed onto the stack is the first one popped off.
- **Push Operation ($O(1)$):** Adds an item to the top of the stack. In recursion, this occurs when a new function call is made.
- **Pop Operation ($O(1)$):** Removes the top item. In recursion, this occurs when a base case is reached and the function returns.
- **Space Complexity:** While recursive algorithms (like calculating Fibonacci or Factorials) can be elegant, they incur an $O(N)$ space complexity overhead due to the call stack, whereas iterative approaches often achieve $O(1)$ space complexity.

2. Advanced Queue Algorithms

Unlike Stacks, Queues operate on a First-In-First-Out (FIFO) principle. Term 2 expands beyond the linear queue to address specific computing bottlenecks.

2.1 Circular Queues

A standard linear queue implemented with an array suffers from false full conditions (where the rear pointer reaches the end of the array, but space exists at the front). Circular Queues solve this by connecting the end of the array back to the front.

Formula for updating the Rear pointer:

$\text{Rear} = (\text{Rear} + 1) \% \text{Maximum_Size}$

Formula for updating the Front pointer:

Front = (Front + 1) % Maximum_Size

2.2 Priority Queues

Elements are dequeued based on priority rather than arrival order. This is highly applicable in modern systems engineering, such as pod scheduling in container orchestration environments, where critical workloads bypass standard queues. Priority Queues are most efficiently implemented using **Heaps** rather than arrays or linked lists.

3. Trees and Traversals

Trees represent hierarchical data. The focus is on Binary Trees, where each node has a maximum of two children (Left and Right).

3.1 Binary Tree Traversals

Traversals dictate the order in which nodes are visited. Understanding the recursive nature of these traversals is critical:

- **In-Order (Left, Root, Right):** Visits the left subtree, then the root, then the right subtree. For a Binary Search Tree (BST), this traversal retrieves elements in sorted, ascending order.
- **Pre-Order (Root, Left, Right):** Used to create a copy of the tree or evaluate expression trees.
- **Post-Order (Left, Right, Root):** Useful for safely deleting a tree, as it ensures children are removed before their parent.

3.2 AVL / Height Balanced Binary Trees

Standard Binary Search Trees can degrade into linked lists ($O(N)$ search time) if unbalanced data is inserted. AVL trees enforce a structural property to guarantee $O(\log N)$ operations.

- **Balance Factor:** $\text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$. An AVL tree requires the balance factor of every node to be strictly -1, 0, or 1.
- **Rotations:** If an insertion violates the balance factor, rotations restructure the tree:
 - *LL (Left-Left) Rotation:* A single right rotation.
 - *RR (Right-Right) Rotation:* A single left rotation.
 - *LR (Left-Right) Rotation:* A left rotation on the child, followed by a right rotation on the parent.
 - *RL (Right-Left) Rotation:* A right rotation on the child, followed by a left rotation on

the parent.

4. Sorting Algorithms and Algorithm Analysis

Algorithm analysis (Big O Notation) provides a theoretical framework to describe how runtime or space requirements scale as the input size (N) grows towards infinity. Term 2 covers six major sorting algorithms.

Algorithm	Mechanism Overview	Best Case (Time)	Worst Case (Time)	Space Complexity
Heap Sort	Builds a Max-Heap, swaps the root (largest) with the last item, reduces heap size, and re-heapifies.	$O(N \log N)$	$O(N \log N)$	$O(1)$
Merge Sort	Divide and conquer. Recursively halves the array, then merges the sorted halves back together.	$O(N \log N)$	$O(N \log N)$	$O(N)$
Quick Sort	Selects a 'pivot' and partitions the array so smaller elements are on the left and larger on the right.	$O(N \log N)$	$O(N^2)$	$O(\log N)$

Algorithm	Mechanism Overview	Best Case (Time)	Worst Case (Time)	Space Complexity
Insertion Sort	Builds the sorted array one item at a time by shifting larger elements to the right. Highly efficient on nearly sorted data.	$O(N)$	$O(N^2)$	$O(1)$
Selection Sort	Iteratively finds the minimum element from the unsorted sublist and swaps it to the front.	$O(N^2)$	$O(N^2)$	$O(1)$

Summary for Optimization: While Merge Sort guarantees a consistent $O(N \log N)$ time complexity, its $O(N)$ space complexity makes it less ideal for memory-constrained environments. Quick Sort is often preferred in practice due to lower overhead and constant space, provided a robust pivot-selection methodology is used to avoid the $O(N^2)$ worst-case scenario.